DYNAMO: Dynamic Skill-Tool Evolution for Vision-Language Agents

Anonymous ACL submission

Abstract

Improving vision-language models (VLMs) on visual reasoning typically requires retraining or hand-designed prompts and tools. We present DYNAMO, a training-free framework in which a frozen VLM agent inspects its own correct and incorrect attempts on a small training subset and evolves two complementary capabilities: reusable reasoning skills for cognitive bottlenecks, and executable visual tools—gated by learned applicability—for perceptual ones, accumulating them in a persistent library without weight updates. Across four visual reasoning benchmarks and six VLM backbones, DYNAMO improves direct inference in 21 of 23 settings (avg. +4.8 acc). The same framework extends to the tool-mastery setting on GTA, where mastery profiling lifts step-level tool selection across all tested backbones. Against task-specific RL (VTool-R1, DeepEyes), DYNAMO matches or approaches RL accuracy at a fraction of the compute (60–94% gap recovery on chart/table; 91.1 vs. 90.1 on V*-7B) and combines additively with RL when available.

1 Introduction

Vision-language models (VLMs) have improved rapidly, yet adapting a frozen VLM to a new visual-reasoning task family still typically requires manually curated SFT data or a hand-designed RL pipeline—per task and per backbone. We ask whether the agent can build its own task-family-specific capability set instead, by inspecting its own behaviour on a small training subset and without any weight updates.

We instantiate this idea in DYNAMO, a training-free framework that evolves two complementary capability types and dynamically equips them at inference. **Skills** are structured Markdown SOPs for *cognitive* bottlenecks, where the visual evidence is available but the agent's reasoning procedure is weak. **Tools** are short Python programs for *perceptual* bottlenecks, where the agent needs a transformed view—a crop, zoom, contrast adjustment, or rerendering—before the evidence becomes legible. On each iteration over a sampled sub-training set, DYNAMO diagnoses both the correct and the incorrect attempts using their images and reasoning traces, proposes multiple candidate skill+tool combinations, and promotes the highest-validation-accuracy candidate into a persistent library. A mastery phase further learns when each tool is safe to invoke, so that promoted capabilities are deployed selectively rather than indiscriminately.

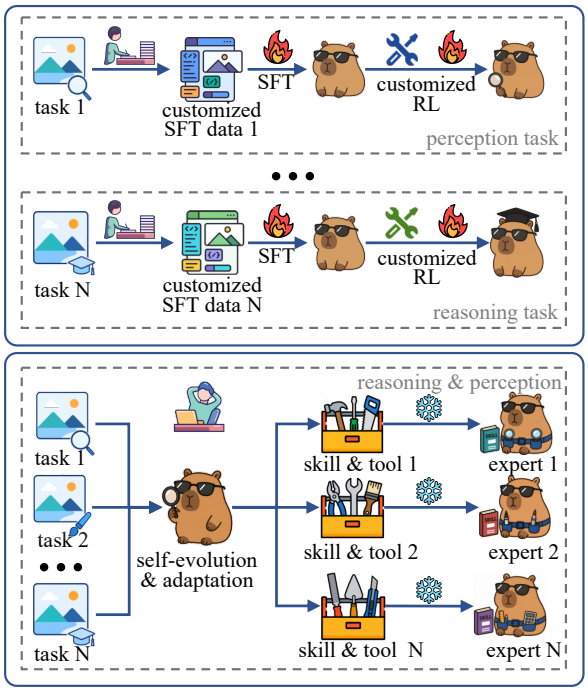

Figure 1: **Labor-free task adaptation via skill-tool evolution.** Per-task VLM adaptation typically requires manually curated SFT data with a hand-designed RL pipeline for every new task family. DYNAMO instead lets a frozen VLM agent evolve its own skill and tool set per task family without human supervision, and dynamically equips the matching set when a case from that family arrives.

We evaluate three questions. **(I) Does DY-**

DYNAMO work across diverse benchmarks and backbones? On ChartQA, MathVista, HRBench, and V* across six VLM backbones, skill-tool co-evolution improves direct inference in 21 of 23 model–benchmark settings, averaging +4.8 accuracy points. **(II) Does the framework extend to a tool-mastery setting with a curated tool set?** On GTA, mastery profiling lifts step-level tool selection, argument, and instruction-following accuracy across all tested backbones. **(III) Can DYNAMO match task-specific RL across different task families?** Across three task families—chart, table, and high-resolution perception—we compare DYNAMO to a representative RL method on each (VTool-R1 for chart/table; DeepEyes for perception). DYNAMO matches or approaches RL accuracy at a fraction of the compute (60–94% gap recovery on chart/table; 91.1 vs. 90.1 on V*-7B perception), and combines additively with RL when both are available.

Contributions.

(1) A **problem formulation** that recasts per-task VLM adaptation as labor-free capability evolution: a frozen agent builds its own task-family-specific skill and tool set from a small training subset, replacing manual SFT data curation and task-specific RL pipelines.

(2) A **multi-candidate evolution loop with mastery** that diagnoses both correct and incorrect attempts, generates candidate skill+tool combinations, selects the best by validation accuracy, and learns per-tool applicability boundaries.

(3) **Empirical evidence** across four visual-reasoning benchmarks, six VLM backbones, the GTA tool-mastery benchmark, and three task families (chart, table, perception) each compared against its representative RL method, showing that capability evolution is a viable training-free alternative—and add-on—to task-specific RL.

2 Related Work

Self-improving agents. A growing line of work lets LLM agents improve from experience without gradient updates: Reflexion (Shinn et al., 2023) keeps a verbal reflection buffer that resets between episodes; Voyager (Wang et al., 2024a) and ExpeL (Zhao et al., 2024) accumulate persistent skill libraries in game and tool-use domains; AutoManual (Zhao et al., 2024) and EvolveR (Wu et al., 2025b) refine instruction manuals or reasoning strategies via self-play; Trace2Skill (**?**) distils trajectory-local lessons into transferable agent skills. Whether language models can bootstrap reasoning capabilities entirely without external scaffolding remains debated (**?**), and broader surveys map the design space for personalised LLM-powered agents (**?**). All these methods operate in text, code, or game domains, never generating executable visual processing nor deciding whether a bottleneck is cognitive or perceptual. DYNAMO extends the persistent-library paradigm into the visual domain with a multimodal diagnostic that text-only frameworks cannot perform.

Tool creation for language models. A complementary line generates new tools on demand: CREATOR (Qian et al., 2023) prompts an LLM to write Python helpers for problems it cannot solve directly; LATM (Cai et al., 2024) and ToolLLM (Qin et al., 2024) scale this to large API collections. These methods generate tools in textual or mathematical domains and do not profile when a generated tool succeeds or fails—a gap that matters for visual tools, where the same operation can help on most images but hurt on specific patterns (e.g. a sharpening filter degrading low-contrast charts). DYNAMO's mastery phase profiles each tool on a held-out case set to characterise its applicability before promotion.

Visual reasoning with tools. Several recent systems equip VLMs with visual processing tools. VTool-R1 (Wu et al., 2025a), DeepEyes (Zheng et al., 2025), Adaptive-CoF (Zhang et al., 2025), PixelReasoner (Su et al., 2025), and V-Thinker (Qiao et al., 2025) all rely on SFT and/or RL with curated trajectories to teach a VLM when and how to invoke a fixed tool inventory. ZoomEye (Shen et al., 2025) and ReFocus (Fu et al., 2025) are training-free but assume a fixed zoom/search algorithm or visual-editing interface; Lever LM (**?**) configures in-context example sequences to leverage pretrained VLMs without retraining. DYNAMO instead generates and validates the skills and tools themselves from the agent's own behaviour on a small training subset, with no weight updates and no predefined tool inventory.

Tool-augmented VLMs with predefined tool sets. VisProg (Gupta and Kembhavi, 2023), ViperGPT (Surís et al., 2023), and Chameleon (Lu et al., 2023) compose hand-curated tool libraries

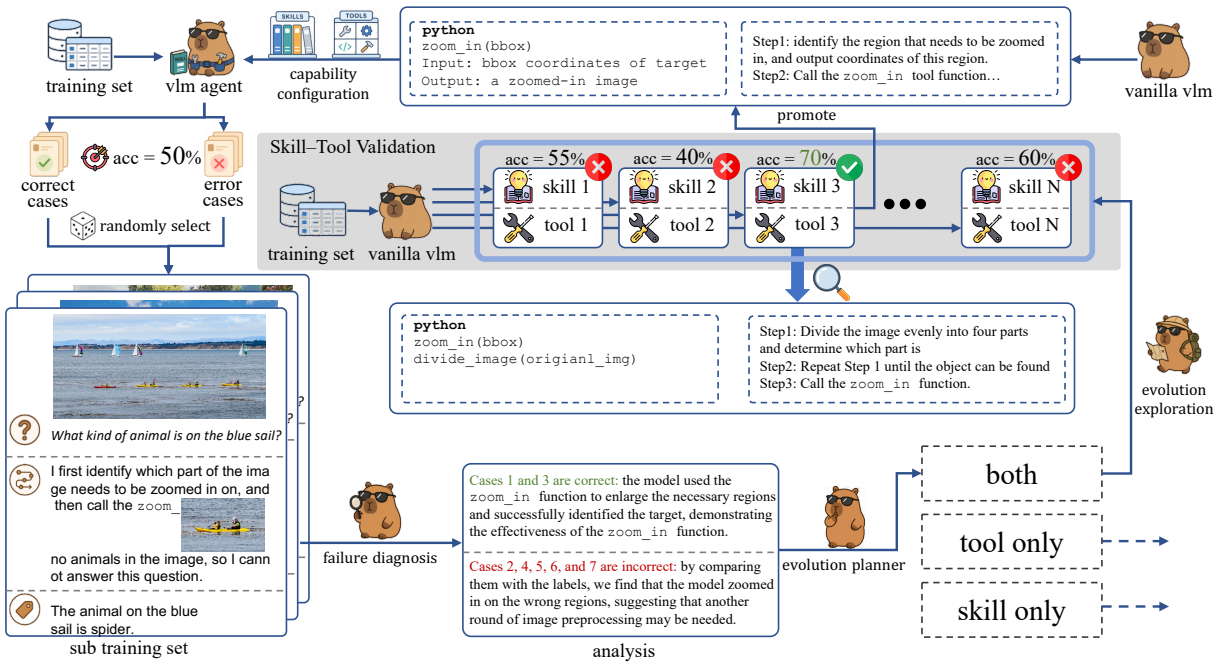

Figure 2: **DYNAMO evolution loop.** On a sampled sub-training set, DYNAMO diagnoses both correct and incorrect attempts, explores candidate skill+tool capabilities, and promotes the best by validation into a persistent library.

(detectors, OCR, arithmetic modules) via program synthesis; MMR-Bench (**?**) extends evaluation of such systems to multimodal LLM routing across diverse backbones and tools. They handle broad compositional visual tasks effectively but cannot recover when the required tool is missing from the human-designed inventory, and cost-aware alignment frameworks such as EcoAlign (**?**) highlight the practical pressure to adapt LVLMs efficiently. DYNAMO fills this gap by generating new tools when the current set is insufficient, bridging fixed-tool composition and training-based tool learning.

3 Method

3.1 Problem Formulation

Let $\mathcal{D}_{\text{train}} = \{(x_i, y_i, \mathbf{v}_i)\}_{i=1}^{k}$ denote a small training subset of k cases, where x_i is the question, y_i the ground-truth answer, and $\mathbf{v}_i$ the associated visual input (one or more images). Let π_θ be a frozen VLM backbone. We define a capability set $\mathcal{C} = (\mathcal{S}, \mathcal{T})$, where $\mathcal{S}$ is a library of *skills* (structured reasoning SOPs) and $\mathcal{T}$ is a library of *tools* (executable Python programs). The agent decomposes solving into a retrieval step and a reasoning step:

$$f_{\mathcal{C}}(x, \mathbf{v}; \pi_\theta) = \pi_\theta(\cdot \mid x,\ \mathbf{v},\ \text{Retrieve}(x, \mathbf{v}; \mathcal{C}))\,, \tag{1}$$

where Retrieve returns the subset of skills and tools relevant to the current input. Given a held-out validation set $\mathcal{D}_{\text{val}}$, our goal is to learn $\mathcal{C}$ that maximises agent accuracy without any weight update:

$$\mathcal{C}^\star = \arg\max_{\mathcal{C}}\ \text{Acc}(f_{\mathcal{C}}; \mathcal{D}_{\text{val}})\,, \tag{2}$$

where the maximisation is over $\mathcal{C}$ only and π_θ remains frozen ($\nabla_\theta = 0$).

3.2 Evolution Loop

Each evolution iteration over $\mathcal{D}_{\text{train}}$ has three phases (Figure 2).

Diagnose. The agent (Eq. 1) solves $\mathcal{D}_{\text{train}}$ with the current $\mathcal{C}$, then samples a sub-training set $\mathcal{D}_{\text{sub}} \subseteq \mathcal{D}_{\text{train}}$ containing both correct and incorrect attempts. The ANALYZERDECIDER inspects $\mathcal{D}_{\text{sub}}$—the questions, original images $\mathbf{v}_i$, reasoning traces τ_i, and any intermediate tool outputs—and emits a root-cause analysis grounded in visual evidence, an identified bottleneck type (*cognitive* when the visual evidence is available but the reasoning procedure is weak, or *perceptual* when the agent needs a transformed visual input), and an action $a \in \{\texttt{skill}, \texttt{tool}, \texttt{both}\}$. Visual input here is essential: deciding whether a crop is misaligned, whether labels are legible, or whether a processed image introduced artefacts requires inspecting the image, not just the reasoning trace.

Explore. Conditioned on the diagnosis, the GENERATOR proposes M candidate skill+tool

combinations, each addressing the diagnosed bottleneck with a different strategy or implementation.

Validate and promote. Each candidate is evaluated on $\mathcal{D}_{\text{train}}$ and the highest-accuracy combination is promoted:

$$m^\star = \arg\max_{m\in\{1,\dots,M\}} \mathrm{Acc}\big(f_{\mathcal{C}\cup(s_m,t_m)};\, \mathcal{D}_{\text{train}}\big)\,,$$
$$\mathcal{C} \leftarrow \mathcal{C} \cup \{(s_{m^\star}, t_{m^\star})\}. \tag{3}$$

Eq. 3 approximates the validation objective in Eq. 2 by its training-set counterpart, and subsumes both an *origin* requirement (the new capability must improve over the previous $f_{\mathcal{C}}$ on the cases it targets) and a *regression* requirement (it must not degrade currently-correct cases). When the action is `both`, the paired skill additionally serves as the tool's mastery SOP, gating tool invocation at inference (Section 3.4). The full procedure runs for N iterations (default N=3); pseudocode is given in Appendix A (Algorithm 1).

3.3 Skills

A *skill* is a structured Markdown document with four fields: **When-to-Use** (a trigger predicate over question and image type), **Strategy** (a numbered step-by-step SOP), **Common Pitfalls**, and **Worked Example**. The Generator writes a skill conditioned on the AnalyzerDecider's root-cause analysis; if a skill covering the same problem class already exists in $\mathcal{S}$, new insights are merged into it rather than creating a duplicate, preventing library bloat. At inference, the Solver retrieves the top-K skills by BM25 similarity to the question and appends them to the system prompt.

3.4 Tools and Mastery

A *tool* is a Python function ($\leq$150 lines) that takes one or more image paths and returns processed images or extracted data, using standard CV libraries (OpenCV, PIL, NumPy). The Generator writes a tool conditioned on a perceptual bottleneck diagnosis; representative examples produced in our experiments include chart re-rendering with enlarged axis labels, saliency-guided sub-region extraction, and contrast enhancement for low-quality documents.

Mastery as a paired skill. When the AnalyzerDecider's action is `both`, the Generator emits a skill alongside the tool, and this paired skill *is* the tool's mastery SOP. Its When-to-Use predicate specifies the input patterns on which the tool helps, its Common Pitfalls flag the patterns on which it hurts, and its Strategy instructs the Solver how to invoke the tool and consume its output. Because tool invocation is gated by retrieving the paired skill, deployment is selective by construction rather than indiscriminate—a property that distinguishes DYNAMO from prior tool-creation methods (Qian et al., 2023; Cai et al., 2024) that expose generated tools without learned applicability boundaries.

External tool sets (Mode B). When a curated tool set $\mathcal{T}_0 \neq \emptyset$ is already provided, DYNAMO can operate in **Mode B**: skip tool generation and instead synthesise a mastery skill for each provided tool $t_j \in \mathcal{T}_0$, learning when to invoke it from the agent's own behaviour on $\mathcal{D}_{\text{train}}$. Mode B requires no code generation, making it practical when tool quality is already high but deployment strategy is unknown. Experiment II (Section 4.3) evaluates this mode.

3.5 Online Adaptation to Distribution Shift

In real deployments, DYNAMO runs against a streaming query distribution that can shift over time—a stream may begin with high-resolution perception cases and later start including MathVista-style reasoning cases, or vice versa. We design DYNAMO to detect such shifts and *update* its skills and tools online, without weight updates and without the practitioner having to anticipate the shift or prepare a per-family training set.

DYNAMO continuously samples a sub-training set from the most recent queries and runs the evolution loop of Section 3.2 on them, accumulating skills and tools into the library $\mathcal{C}$. Let W be a rolling-window length. The dominant task family at step t is

$$\hat{f}_t = \arg\max_{f} \sum_{i=t-W+1}^{t} \mathbf{1}[\,\mathrm{fam}(x_i) = f\,]\,, \tag{4}$$

where $\mathrm{fam}(\cdot)$ classifies each query from its question and image features. When Eq. 4 flips ($\hat{f}_t \neq \hat{f}_{t-1}$), the next evolution iteration is triggered on the new sub-stream and produces updated skills and tools that handle the new distribution. The library is thus kept in sync with the current query distribution without any human intervention. We treat this autonomous online adaptation as one of the method's contributions; Experiment I (Section 4.2) evaluates it against a static baseline that never refreshes and an oracle whose full capability set is available from the start.

4 Experiments

4.1 Experimental Setup

Benchmarks. For the evolution-from-scratch setting (Exp I) we use four visual reasoning benchmarks: **ChartQA** (Masry et al., 2022) (multi-step numerical reasoning over charts; 2,500 test questions, relaxed accuracy with 5% tolerance), **MathVista** (Lu et al., 2024) (math reasoning grounded in figures and plots; 1,000 testmini questions, accuracy), **HRBench** (Wang et al., 2025) (perception on $\geq$4K-resolution images; accuracy), and **V*** (Wu and Xie, 2024) (visual search for an object's property; 191 questions, accuracy); the first two stress *cognitive* bottlenecks while the latter two stress *perceptual* ones. For the tool-mastery setting (Exp II) we use **GTA** (Wang et al., 2024b), a benchmark of 229 real-world tasks with tools across perception, operation, logic, and creativity categories. GTA reports step-by-step accuracy on tool selection, argument prediction, and instruction following, plus an end-to-end answer accuracy. For the RL comparison (Exp III) we follow the VTool-R1 protocol (Wu et al., 2025a) on its **Chart** and **Table** reasoning splits and reuse **V*** and **HRBench4K** as the high-resolution splits of the DeepEyes protocol (Zheng et al., 2025).

Foundation models. Experiment I evaluates five proprietary and open-weight VLMs: **GPT-4o**, **o4-mini**, **GPT-5.4**, **Doubao-Seed-2.0**, and **Qwen3.5-27B**. Experiment II evaluates **GPT-4o**, **GPT-5.4**, and **Doubao-Seed-2.0** on the GTA benchmark, plus a controlled **Doubao-Seed-2.0-Pro** analysis that inspects how provided-tool skills change with tool abstraction and task environment. Experiment III uses the **Qwen2.5-VL** family (3B, 7B, 32B) to align with the VTool-R1 evaluation protocol. All VLM weights are *frozen* throughout; no gradient updates are performed.

Evolution protocol. For each benchmark, we sample 20% of the official training split as $\mathcal{D}_{\text{train}}$ and run the DYNAMO evolution loop for $N{=}3$ iterations. The capability set $\mathcal{C}$ is then frozen and evaluated on the held-out validation / test split. We report the average over three independent random seeds (different 20% subsets) to account for sampling variance where repeated runs are available.

Baselines. For Experiment I, we compare four evolution configurations that isolate the contribution of each component: **None (Base Agent)** invokes the frozen VLM with no evolved capabilities (the zero-shot baseline); **Skill Only** restricts the Generator to producing reasoning skills (no tool generation); **Tool Only** restricts the Generator to producing executable visual tools (no skill generation); **Full** is the unrestricted DYNAMO system, where the AnalyzerDecider chooses on each case whether to generate a skill, a tool, or both.

Evaluation metrics. All primary metrics are task-level accuracy on the held-out split. For Experiment II we report four of GTA's five canonical metrics: three step-by-step metrics—**InstAcc** (Instruction-Following Accuracy: fraction of steps executed without errors), **ToolAcc** (Tool Selection Accuracy), and **ArgAcc** (Argument Prediction Accuracy)—and the end-to-end **AnsAcc** (Answer Accuracy on full tool-using execution). For the supplementary controlled GTA analysis, we also report tool usage rate, average number of tool calls, and dominant tool-call chains. Experiment III follows the VTool-R1 evaluation protocol (Wu et al., 2025a) on Chart and Table reasoning splits, and uses DeepEyes (Zheng et al., 2025) as the RL reference for high-resolution V* and HRBench4K visual-search tasks.

4.2 Experiment I: Autonomous Evolution from Scratch

We test whether DYNAMO can bootstrap a useful capability library from scratch ($\mathcal{S}{=}\emptyset$, $\mathcal{T}{=}\emptyset$) across five backbones and four benchmarks, and whether the library remains useful when the query distribution shifts over time (Section 3.5).

Per-benchmark results. Table 1 reports accuracy for the four configurations on each backbone. Comparing the Full system to the zero-shot *None* baseline, DYNAMO improves direct inference on all 20 model–benchmark cells, with an average gain of $+4.7$ accuracy points. The largest gains are on o4-mini / HRBench ($+12.7$), GPT-5.4 / HRBench ($+10.7$), Doubao-Seed-2.0 / V* ($+8.8$), and Qwen3.5-27B / HRBench ($+8.2$). The relative strength of *Skill Only* and *Tool Only* also follows the cognitive / perceptual split: Skill Only is competitive on ChartQA and MathVista, where the gain comes from better decomposition and calculation procedures, while Tool Only and Full dominate on HRBench and V*, where sub-region inspection and visual search are central. On a few cells (GPT-4o / MathVista, Doubao / V*) Skill Only or Tool Only slightly edges out Full—an expected consequence

Model	Evolution Strategy	HRBench	MathVista	V*	ChartQA
GPT-4o	None (Base Agent)	0.6386	0.6711	0.5828	0.7552
	Skill Only	0.6733±0.0121	0.7122±0.0038	0.6424±0.0066	0.7785±0.0026
	Tool Only	0.6452±0.0044	0.6900±0.0029	0.6225±0.0066	0.7722±0.0029
	Full	0.6886±0.0071	0.7189±0.0029	0.6689±0.0239	0.7799±0.0045
o4-mini	None (Base Agent)	0.73	0.753333	0.72847	0.7833
	Skill Only	0.7724±0.0058	0.7470±0.0107	0.7616±0.0066	0.7866±0.0151
	Tool Only	0.7457±0.1670	0.7356±0.0328	0.8387±0.0629	0.7920±0.0213
	Full	0.8571±0.0099	0.7559±0.0055	0.8698±0.0076	0.8007±0.0081
gpt5.4	None (Base Agent)	0.7471	0.7589	0.7748	0.7974
	Skill Only	0.7805±0.0036	0.7752±0.0039	0.7682±0.0199	0.8090±0.0022
	Tool Only	0.8367±0.0346	0.7663±0.0061	0.8477±0.0132	0.8108±0.0052
	Full	0.8538±0.0036	0.7815±0.0101	0.8521±0.0233	0.8276±0.0278
Doubao-Seed-2.0	None (Base Agent)	0.8214	0.8544	0.8675	0.8152
	Skill Only	0.8600±0.0049	0.8556±0.0109	0.8609±0.0115	0.8198±0.0031
	Tool Only	0.9005±0.0022	0.8659±0.0061	0.9448±0.0038	0.8156±0.0068
	Full	0.8990±0.0064	0.8681±0.0105	0.9558±0.0038	0.8347±0.0295
Qwen3.5-27B	None (Base Agent)	0.8171	0.8088	0.88079	0.8114
	Skill Only	0.8721±0.0030	0.8152±0.0080	0.8675±0.0066	0.8097±0.0090
	Tool Only	0.8986±0.0000	0.8241±0.0034	0.9514±0.0101	0.8115±0.0041
	Full	0.9010±0.0008	0.8252±0.0023	0.9603±0.0175	0.8158±0.0026

Table 1: **Autonomous evolution from scratch.** Accuracy across five VLM backbones and four benchmarks. The Full DYNAMO configuration improves over the zero-shot *None* baseline on all 20 model–benchmark cells.

of leaving capability choice to the AnalyzerDecider rather than enforcing both per case.

Capability quality. Appendix D inspects three representative forms of evolved capability: a ReFocus-style chart/table editing case, a ZoomEye-style coarse-to-fine search case, and an interleaved skill that pairs a textual SOP with visual evidence. The case studies illustrate what DYNAMO generates; they do not claim to reproduce the full algorithms of ReFocus (Fu et al., 2025) or ZoomEye (Shen et al., 2025).

Online adaptation to distribution shift. To simulate real online user traffic—where the task distribution grows and shifts over time as different users and requests arrive—we replay a 208-step non-stationary stream in which each phase introduces a new task family while keeping carry-over cases from earlier phases: starting with HRBench, then adding MathVista, ChartQA, and finally V* as the new dominant family. The replay reuses completed per-case outputs, isolating the adaptation policy from the cost of regenerating capabilities online. We compare *Direct* (no capabilities), *Static* (capabilities evolved on the initial distribution only and never updated), *Online adapt* (DYNAMO detects the shift and updates skills and tools on the new sub-stream), and *Oracle* (capabilities pre-evolved on every family, an upper bound).

Figure 3 compares the four policies across all five backbones and three stream constructions: the *natural* mixture (a fixed-seed mix of the four families), *capability-relevant* (cases whose outcome depends on the matching capability), and *stress* (cases the matching capability is required to fix). The ordering *Direct* $\leq$ *Static* $<$ *Online adapt* $\approx$ *Oracle* is consistent across every backbone $\times$ protocol cell. On GPT-5.4 the gap is representative: *Online adapt* reaches 0.83 vs. 0.74 static on natural; widens to 0.89 vs. 0.65 on capability-relevant; and on stress, *Direct* collapses to 0.03 while *Online adapt* still recovers to 0.95. The per-step trajectory along the stream, with shift-detection latencies of 2–7 cases per phase, is shown in Appendix C.3 (Figure 5).

4.3 Experiment II: Learning to Use Pre-Provided Tools

Results on GTA. Table 2 compares three methods on the 121-case GTA validation split: *Base Agent* (zero-shot VLM with the GTA tools exposed); an *XSkill-style* baseline (Jiang et al., 2026) that injects a generic visual-reasoning skill header on top of the same tool environment but without

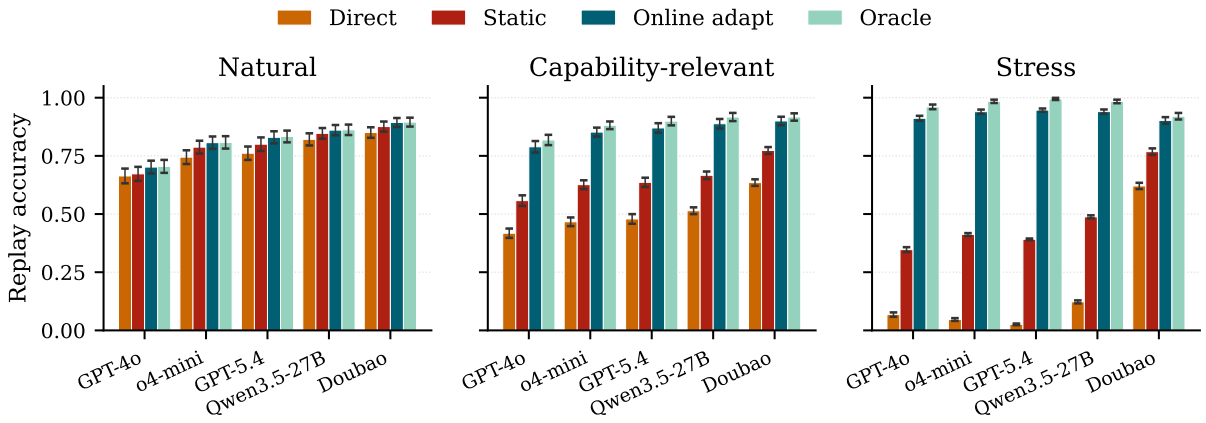

Figure 3: **Online adaptation to distribution shift.** Replay accuracy across five backbones and three stream constructions. *Online adapt* stays close to the *Oracle* upper bound and far above *Static* everywhere; the gap is largest under the stress protocol, where *Direct* nearly collapses. Error bars are seed std (50–200 seeds per cell).

GTA-specific protocols; and DYNAMO Mode B (skills + mastery learned on the training set). DYNAMO improves every metric on every backbone, with the largest gains on Doubao-Seed-2.0 (+18.7 ToolAcc, +15.8 ArgAcc, +16.1 InstAcc, +4.9 AnsAcc) and smaller but uniformly positive gains on GPT-4o (+4.3 / +2.2 / +11.4 / +5.0) and GPT-5.4 (+6.8 / +3.2 / +6.1 / +1.7).

What does environment-specific mastery contribute? The *XSkill-style* baseline isolates generic skill injection from environment-specific mastery: it uses the same tools and scorer but its skill is a generic visual-reasoning prompt without GTA's tool-chain, argument, or answer-normalisation protocols. On every backbone, DYNAMO beats XSkill-style by 11–20 points on each step-level metric and 2.5–4.9 on AnsAcc. More tellingly, *XSkill-style itself drops below Base* on the step-level metrics for GPT-4o and GPT-5.4—adding a generic visual-reasoning prompt without learning the environment's tool protocol can actually confuse tool selection. The gain over both baselines is therefore attributable not to skill injection per se, but to mastery skills that encode the target environment's specific tool chains, argument formats, and answer normalisation rules.

Controlled tool-policy analysis. Table 3 runs a complementary controlled study on GTA with Doubao-Seed-2.0-Pro to inspect what the learned policy actually does when either the tool abstraction or the task environment changes. Two findings stand out. First, fixing the environment to OCR+Calculator, stronger composite tools compress the policy: atomic tools reach 86.7 via an `OCR→Calculator` chain (22/30 cases), while `VisualArithmeticSolver` reaches 90.0 with a one-call policy (30/30). Second, the same `OCR` tool plays different roles in different environments: in OCR+Calc it feeds arithmetic (`OCR→Calculator`, 19/20), while in OCR+Search it feeds external lookup (`OCR→Search→OCR→Search`, 20/20).

Model	Method	ToolAcc	ArgAcc	InstAcc	AnsAcc
GPT-4o	Base Agent	80.3	62.0	52.0	66.1
	XSkill-style	71.3	53.0	49.5	68.6
	DYNAMO (Ours)	**84.6**	**64.2**	**63.4**	**71.1**
GPT-5.4	Base Agent	79.6	60.2	57.0	72.7
	XSkill-style	66.3	48.0	46.2	71.1
	DYNAMO (Ours)	**86.4**	**63.4**	**63.1**	**74.4**
Doubao-Seed-2.0	Base Agent	67.7	45.5	41.6	76.9
	XSkill-style	72.8	48.4	44.4	76.9
	DYNAMO (Ours)	**86.4**	**61.3**	**57.7**	**81.8**

Table 2: **Learning to use pre-provided tools on GTA (121-case validation split).** DYNAMO Mode B (skills + mastery only; no new tool generation) improves every metric on every backbone and beats both *Base Agent* and the *XSkill-style* (Jiang et al., 2026) generic-skill-injection baseline. XSkill-style itself drops below Base on the step-level metrics for GPT-4o and GPT-5.4, showing that generic skill injection without environment-specific protocols can confuse tool selection.

4.4 Experiment III: RL and Self-Evolution Across Visual Tool Tasks

We compare DYNAMO against task-specific RL on three task families using the Qwen2.5-VL family (3B, 7B, 32B): Chart and Table reasoning under the VTool-R1 protocol (Wu et al., 2025a), and high-resolution V* and HRBench4K under the DeepEyes protocol (Zheng et al., 2025). Two cells (32B

Study	Condition	Cases	Acc.	Tool use	Avg. calls
Tool strength	Direct, no tools	30	83.3	0.0	0.00
Tool strength	Atomic OCR+Calc	30	86.7	100.0	1.83
Tool strength	Composite VAS	30	90.0	100.0	1.00
OCR role	OCR+Calc env	20	80.0	100.0	2.20
OCR role	OCR+Search env	20	85.0	100.0	2.35

Table 3: **Controlled tool-policy analysis on GTA (Doubao-Seed-2.0-Pro).** Top: stronger composite tools compress the policy at higher accuracy. Bottom: the same `OCR` tool exhibits different downstream behavior in different environments (dominant call chains discussed in prose). Acc./Tool use in %; Avg. calls is a count.

Backbone	Variant	Chart	Table	V*	HRBench4K
Qwen2.5-VL-3B	Direct	45.3995	43.0921	71.20	63.62
	RL	66.95	56.25	78.01	66.50
	DYNAMO	59.56	51.97	84.29	70.50
	DYNAMO + RL	68.28	59.54	85.34	71.25
Qwen2.5-VL-7B	Direct	56.0532	56.9078	79.06	70.50
	RL	75.18	68.42	84.82	74.38
	DYNAMO	75.060	67.43	85.86	75.12
	DYNAMO + RL	76.88	69.41	92.67	81.25

Table 4: **Self-evolution vs. task-specific RL across three task families.** Accuracy (%). DYNAMO alone matches or beats RL on all six high-resolution cells and recovers most of the Chart/Table gap; DYNAMO + RL is additive in nearly every cell. Blank cells are runs not yet completed.

DYNAMO + RL on Chart and Table) are runs not yet completed.

Chart and Table reasoning. Using the gap-recovery fraction (DYNAMO − Direct)/(RL − Direct), DYNAMO recovers 65.7% and 99.4% of the Chart gap at 3B and 7B respectively, and *exceeds* RL at 32B (86.4 vs. 84.3). On Table, recovery is 67.5% / 91.4% / 46.8%. DYNAMO + RL further beats RL alone by 1–3 points wherever the row is reported, indicating that the two interventions compose additively.

High-resolution visual search. On V* and HRBench4K, DYNAMO alone matches or beats task-specific RL at every scale: it gains +6.3 / +1.0 / ±0.0 over RL on V* (3B / 7B / 32B) and +4.0 / +0.7 / +5.1 on HRBench4K. Adding DYNAMO on top of the RL-trained policy raises 7B V* to 92.7 and 7B HRBench4K to 81.3, the strongest configurations in those cells.

Cost. DYNAMO performs no weight updates: an evolution run uses 20% of each benchmark's training split for $N=3$ passes and invokes the frozen VLM for solving, diagnosis, generation, and validation. The comparison therefore shows that across these three task families, DYNAMO recovers most of the RL gain on Chart/Table and matches or beats RL on high-resolution visual search, while composing additively with RL when both are available.

4.5 Ablation Studies

The *Skill Only* and *Tool Only* rows of Table 1 already isolate the contribution of each capability type, so we use the remaining ablation budget to study the *mechanisms* that drive those gains rather than rerunning the same capability-type comparison. We ablate six mechanisms inside the evolution loop: generic prompting vs. evolved capability, failure-only vs. both-correct-and-incorrect diagnosis, text-only vs. multimodal diagnosis, single-candidate vs. multi-candidate exploration, unpaired vs. paired mastery skill at tool invocation, and per-case vs. persistent capability storage. The full mechanism table, per-ablation discussion, and sensitivity curves are deferred to Appendix B for space.

5 Conclusion

DYNAMO is a training-free framework in which a frozen VLM agent builds and updates its own task-family-specific capability library from a small training subset. The agent inspects both correct and incorrect attempts on a sampled sub-training set, an evolution planner decides whether to evolve a reasoning skill, an executable visual tool, or both, and the best of multiple candidates is promoted by validation accuracy. A mastery skill paired with each tool gates its invocation at inference, and an online adaptation loop updates the library when the query distribution shifts.

Across four visual reasoning benchmarks and five VLM backbones, DYNAMO improves direct inference on all 20 model–benchmark cells. On the tool-mastery benchmark GTA, the same framework lifts step-level tool selection, argument, and instruction-following accuracy across every tested backbone. Compared with task-specific RL (VTool-R1, DeepEyes) on chart, table, and high-resolution perception tasks, DYNAMO matches or beats RL on the high-resolution side and recovers most of the gap on the others, while composing additively when RL is also available. Together, these results suggest that a substantial part of task-specific VLM adaptation can be obtained without any gradient up-

dates, by letting the agent shape its own capability library.

Limitations

Short evolution horizon. We report results at $N{=}3$ iterations; the evolution dynamics suggest gains plateau by $N{=}5$, but we have not evaluated whether longer runs (e.g. $N{>}10$) eventually overfit the training distribution.

Tool quality ceiling. Tool effectiveness is bounded by the backbone's Python coding ability. For benchmarks requiring specialised computer-vision operations beyond standard libraries (e.g. learned feature extractors), the Generator may fail to produce a working tool, limiting the perceptual gains.

Benchmark scope. We evaluate four visual-reasoning benchmarks plus GTA and the VTool-R1 / DeepEyes splits. Broader coverage (video QA, 3D spatial reasoning, medical imaging) is needed before strong generality claims about the cognitive / perceptual decision rule.

API cost. Each training case incurs roughly 15K tokens for the combined attempt, diagnosis, generation, and validation pass. While negligible compared with RL training, this cost should be accounted for when comparing training-free methods.

References

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2024. Large language models as tool makers. In *ICLR*. OpenReview.net.

Xingyu Fu, Minqian Liu, Zhengyuan Yang, John Corring, Yijuan Lu, Jianwei Yang, Dan Roth, Dinei A. F. Florêncio, and Cha Zhang. 2025. Refocus: Visual editing as a chain of thought for structured image understanding. In *ICML*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. In *CVPR*, pages 14953–14962. IEEE.

Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. 2026. Xskill: Continual learning from experience and skills in multimodal agents. *CoRR*, abs/2603.12056.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2024. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *ICLR*. OpenReview.net.

Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models. In *NeurIPS*.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq R. Joty, and Enamul Hoque. 2022. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *ACL (Findings)*, Findings of ACL, pages 2263–2279. Association for Computational Linguistics.

Cheng Qian, Chi Han, Yi Ren Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: tool creation for disentangling abstract and concrete reasoning of large language models. In *EMNLP (Findings)*, Findings of ACL, pages 6922–6939. Association for Computational Linguistics.

Runqi Qiao, Qiuna Tan, Minghan Yang, Guanting Dong, Peiqing Yang, Shiqiang Lang, Enhui Wan, Xiaowan Wang, Yida Xu, Lan Yang, Chong Sun, Chen Li, and Honggang Zhang. 2025. V-thinker: Interactive thinking with images. *CoRR*, abs/2511.04460.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *ICLR*. OpenReview.net.

Haozhan Shen, Kangjia Zhao, Tiancheng Zhao, Ruochen Xu, Zilun Zhang, Mingwei Zhu, and Jianwei Yin. 2025. Zoomeye: Enhancing multimodal llms with human-like zooming capabilities through tree-based image exploration. In *EMNLP*, pages 6602–6618. Association for Computational Linguistics.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *NeurIPS*.

Alex Su, Haozhe Wang, Weiming Ren, Fangzhen Lin, and Wenhu Chen. 2025. Pixel reasoner: Incentivizing pixel-space reasoning with curiosity-driven reinforcement learning. *CoRR*, abs/2505.15966.

Dídac Surís, Sachit Menon, and Carl Vondrick. 2023. Vipergpt: Visual inference via python execution for reasoning. In *ICCV*, pages 11854–11864. IEEE.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024a. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024.

Jize Wang, Zerun Ma, Yining Li, Songyang Zhang, Cailian Chen, Kai Chen, and Xinyi Le. 2024b. GTA: A benchmark for general tool agents. In *NeurIPS*.

Wenbin Wang, Liang Ding, Minyan Zeng, Xiabin Zhou, Li Shen, Yong Luo, Wei Yu, and Dacheng Tao. 2025. Divide, conquer and combine: A training-free framework for high-resolution image perception in multimodal large language models. In *AAAI*, pages 7907–7915. AAAI Press.

Mingyuan Wu, Jingcheng Yang, Jize Jiang, Meitang Li, Kaizhuo Yan, Hanchao Yu, Minjia Zhang, Chengxiang Zhai, and Klara Nahrstedt. 2025a. Vtool-r1: Vlms learn to think with images via reinforcement learning on multimodal tool use. *CoRR*, abs/2505.19255.

Penghao Wu and Saining Xie. 2024. V*: Guided visual search as a core mechanism in multimodal llms. In *CVPR*, pages 13084–13094. IEEE.

Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2025b. Evolver: Self-evolving LLM agents through an experience-driven lifecycle. *CoRR*, abs/2510.16079.

Xintong Zhang, Zhi Gao, Bofei Zhang, Pengxiang Li, Xiaowen Zhang, Yang Liu, Tao Yuan, Yuwei Wu, Yunde Jia, Song-Chun Zhu, and 1 others. 2025. Adaptive chain-of-focus reasoning via dynamic visual search and zooming for efficient vlms. *arXiv preprint arXiv:2505.15436*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: LLM agents are experiential learners. In *AAAI*, pages 19632–19642. AAAI Press.

Ziwei Zheng, Michael Yang, Jack Hong, Chenxiao Zhao, Guohai Xu, Le Yang, Chao Shen, and Xing Yu. 2025. Deepeyes: Incentivizing "thinking with images" via reinforcement learning. *CoRR*, abs/2505.14362.

A Implementation Details

Algorithm. Algorithm 1 gives the full DYNAMO evolution procedure described in Section 3.2.

Algorithm 1: DYNAMO evolution loop.

Input: $\mathcal{D}_{\text{train}}$, frozen VLM π_θ, iterations N, candidates M
Output: Capability set $\mathcal{C} = (\mathcal{S}, \mathcal{T})$

$\mathcal{C} \leftarrow (\emptyset, \emptyset)$;
for $n = 1$ **to** N **do**
  Solve $\mathcal{D}_{\text{train}}$ with $f_{\mathcal{C}}$; collect attempts;
  $\mathcal{D}_{\text{sub}} \leftarrow$ sample from $\mathcal{D}_{\text{train}}$;
  $a, r \leftarrow$ ANALYZERDECIDER$(\mathcal{D}_{\text{sub}}; \pi_\theta)$;
  $\{(s_m, t_m)\}_{m=1}^{M} \leftarrow$ GENERATOR$(a, r; \pi_\theta)$; `// $s_m$ paired with $t_m$ acts as its mastery SOP`
  $m^\star \leftarrow \arg\max_m \text{Acc}\big(f_{\mathcal{C}\cup(s_m,t_m)}; \mathcal{D}_{\text{train}}\big)$;
  if *accuracy improves* **then**
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{(s_{m^\star}, t_{m^\star})\}$;
  end
end
return $\mathcal{C}$;

Role prompts. DYNAMO uses three role-specific system prompts for the same base VLM: *Solver* (standard QA prompt with capability context injected), *AnalyzerDecider* (root-cause analysis and decision), and *Generator* (skill or tool generation). All prompts are provided in full at `[anonymised repository]`.

Skill retrieval. At inference time, the Solver retrieves the top-K=3 skills from $\mathcal{S}$ using BM25 cosine similarity over skill titles and When-to-Use fields. Retrieved skill text is appended to the system prompt.

Tool invocation. The Solver inspects each tool's mastery SOP to decide whether to invoke it on the current input. If the input matches a tool's supported patterns, the tool is applied and its output image(s) are appended to the multimodal context. Tool execution is sandboxed with a 30-second timeout and resource limits.

Hyperparameters. Table 5 lists all hyperparameter settings.

Table 5: Hyperparameter settings.

Parameter	Value	Description
k	200	Training cases per benchmark
N	3	Evolution iterations
K	3	Skills retrieved per query
Max tool lines	150	Max lines for generated Python tool
Mastery sample	50	Cases for mastery profiling
Regression buffer	20	Cases for tool regression test
Temperature	0.7	Generator temperature
Max tokens (gen.)	2048	Max tokens for capability generation

Compute. Experiments use the frozen VLM backbones listed in Section 4.1. Total API calls per benchmark: $\leq k \times N \times 3 = 1{,}800$ calls (solver + analyzer + generator, upper bound). Estimated token usage: $\sim$3M tokens per benchmark. No GPU training is performed.

B Mechanism Ablations

Experiment I in the main paper already isolates the contribution of each capability *type*: the *None*, *Skill Only*, *Tool Only*, and *Full* rows of Table 1 answer whether skills, tools, or both account for the observed gains. The ablations here test a complementary question: when the Full system is used, which *mechanisms* inside the evolution loop (Section 3.2) are doing the work? Each variant removes one design decision and holds the rest fixed, so each row of Table 6 rules out one alternative explanation for the Full system's gains.

Setup. All variants use the same protocol as Experiment I (frozen GPT-5.4 backbone; 20% of each benchmark's training split as $\mathcal{D}_{\text{train}}$; N=3 evolution iterations; M candidate skill+tool combinations per iteration) and rebuild $\mathcal{C}$ from scratch so any deficit reflects the ablated mechanism rather than a carried-over artifact. The two primary benchmarks are **ChartQA** (cognitive / procedural) and **HRBench** (perceptual / high-resolution); **MathVista** and **V*** are reported as secondary checks.

Main mechanism table. Table 6 reports six ablation variants spanning the four mechanism families: diagnosis (whether the AnalyzerDecider sees correct cases and visual input), exploration (multi-candidate generation), inference-time gating (paired mastery skill), and persistence (cross-case library). The Full row reproduces the Full entry from Table 1 for reference.

Variant	Ablated mechanism	ChartQA	HRBench	MathVista	V*
Full DYNAMO	— (full evolution loop)				
Generic Skill	evolved library → hand-neutral task prompt				
Failure-Only Diagnosis	AnalyzerDecider sees only incorrect cases				
Text-only Diagnosis	AnalyzerDecider sees no images or tool artefacts				
Single-Candidate (M=1)	one Generator proposal per iteration; no selection				
No Paired Mastery Skill	tool emitted without its paired skill (action = both)				
No Persistence	capabilities not stored across cases				

Table 6: **Mechanism ablations.** Each row removes one mechanism from DYNAMO while holding the rest fixed (GPT-5.4; 20% training split; N=3). The Full row reproduces the Full entry from Table 1. Numbers will be filled in once the corresponding runs complete.

B.1 Generic Skill vs. Evolved Capability

What is removed. The evolved capability library is replaced with a single hand-neutral task prompt that gives generic task advice (for ChartQA: "read the chart carefully, identify relevant values, reason step by step"; for HRBench: "inspect local details before answering and verify the target object"). No diagnosis, generation, validation, or persistence is run.

What this rules out. A pure prompt-engineering account would predict that Generic Skill closes most of the gap to Full, since the evolved skills are themselves natural-language SOPs. If Generic Skill improves over Direct by only a small margin and stays well below Full, the gap is attributable to mechanism-level evolution—failure-specific procedures, applicability triggers, and pitfalls that a generic prompt cannot anticipate—rather than to the presence of any task-aware prompt.

B.2 Failure-Only Diagnosis

What is removed. The AnalyzerDecider only sees the incorrect cases in the sampled sub-training set; the correct cases are dropped. Everything else—multimodal input, multi-candidate exploration, paired mastery skill, persistence—is unchanged.

What this rules out. Most prior self-improvement work treats correct cases as nothing to learn from. This variant tests that assumption directly. If Failure-Only Diagnosis matches Full, then the correct cases carry no useful diagnostic signal and the abstract's framing of "diagnoses both correct and incorrect attempts" is decorative. A gap supports the design choice: correct attempts tell the AnalyzerDecider *which sub-patterns the current $\mathcal{C}$ already handles*, which sharpens the next capability proposal toward what is still missing.

B.3 Text-only Diagnosis

What is removed. The AnalyzerDecider's inputs are restricted to text only: the question, the agent's answer (correct or incorrect), the ground-truth answer, and the reasoning trace. No original image, no intermediate tool output, and no processed visual artefact is passed in.

What this rules out. Text reflection can say *that* an answer was wrong, but it cannot tell whether a crop was misaligned, whether a zoomed region missed the target, or whether a processed image introduced artefacts. We expect the gap to Full to be largest on the perceptual benchmarks (HRBench, V*). On ChartQA the gap should be smaller because many failures there are procedural; a small ChartQA gap with a large HRBench/V* gap is itself the signature of the visual-diagnosis claim.

B.4 Single-Candidate (M=1)

What is removed. The Generator produces a single capability proposal per iteration (the same setting as typical generate-then-validate tool-creation methods such as CREATOR (Qian et al., 2023) and LATM (Cai et al., 2024)). The validate-and-promote step (Eq. 3) collapses to a binary accept/reject of that single candidate against the training set.

What this rules out. A reviewer might ask whether the multi-candidate $\arg\max$ in Eq. 3 is load-bearing or whether the first proposal is usu-

ally good enough. If Single-Candidate roughly matches Full, the exploration step is decorative. A measurable gap shows that diverse candidates plus selection by training-set accuracy is what turns a noisy proposal distribution into a reliably improving library.

B.5 No Paired Mastery Skill

What is removed. When the AnalyzerDecider's action is `both`, the Generator emits only the tool and not its paired mastery skill. The tool is added to $\mathcal{T}$ but has no WHEN-TO-USE predicate gating its invocation at inference, so any skill retrieval that surfaces a tool reference exposes the tool unconditionally.

What this rules out. Prior tool-creation methods deploy generated tools indiscriminately. The paired-skill design in Section 3.4 predicts that gating tool invocation by retrieval of an applicability-encoding skill is what keeps tool usage selective. No Paired Mastery Skill is expected to keep or even increase tool-usage rate but reduce accuracy on benchmarks where naive tool exposure can hurt—MathVista is the canonical cautionary case.

B.6 No Persistence

What is removed. Capabilities promoted during one iteration are not stored in $\mathcal{C}$. The agent can still reflect on a case and immediately retry it with the generated capability, but the capability is discarded before the next case is processed, so no cross-case retrieval occurs.

What this rules out. A retry-only account would predict that most of Full's gain comes from in-place retry on the triggering case rather than from cross-case reuse. If No Persistence matches Full on held-out accuracy, the library is a bookkeeping device with no inferential value. A gap shows that the system's advantage comes from accumulating capabilities that future cases can retrieve, not just from one more attempt at the triggering case. ChartQA is the cleanest test because failures there form recurring families (stacked-bar boundary reading, multi-series selection) that a single capability can fix across many held-out cases.

B.7 Sensitivity to Training Subset Size and Iteration Count

The main paper fixes the training subset at 20% of each benchmark's training split and runs $N{=}3$ evolution iterations. A sensitivity sweep over smaller fractions (e.g. 5%, 10%) and over $N \in \{1, 3, 5\}$ tests how quickly the library saturates: a small subset should already expose the most frequent procedural / perceptual patterns, with diminishing returns as the subset grows; $N{=}1$ should miss capabilities that only surface after earlier ones reshape the trajectory, while $N{=}3$ and $N{=}5$ should produce comparable libraries. We treat this as a robustness check rather than a tuning study; the operating point is chosen to fit the per-benchmark budget.

B.8 Summary

Read together, the six mechanism ablations are designed to triangulate the same claim from six different angles. Generic Skill rules out a pure prompt-engineering explanation; Failure-Only Diagnosis rules out "correct cases carry no signal"; Text-only Diagnosis rules out "text reflection is enough" and supports the visual-diagnosis channel; Single-Candidate rules out "the first proposal is good enough" and supports multi-candidate exploration; No Paired Mastery Skill rules out "expose every generated tool" and supports applicability-bounded deployment; and No Persistence rules out "per-case retry is enough" and supports cross-case capability accumulation. The expected pattern is that removing any one mechanism reduces held-out accuracy noticeably, but no single ablation collapses the system—consistent with the design view that the six mechanisms implement complementary aspects of capability evolution rather than redundant copies of the same underlying lever.

Practical diagnostic. The mechanism ablations also suggest a practical rule of thumb for new task families. If the bottleneck is procedural—visual evidence is present but the reasoning is weak—skill evolution is the lower-cost first choice; if a different view of the image is required before the evidence becomes accessible, tool evolution is necessary. A practitioner targeting a new benchmark can use a small pilot run (e.g. 5% of the training split with $N{=}1$) to inspect which kinds of capabilities are generated and whether they transfer beyond the triggering examples.

C Additional Results

C.1 Training Set Size Sensitivity

Figure 4 shows ChartQA accuracy as a function of training set size $k \in \{10, 25, 50, 100, 200\}$ at $N{=}3$ iterations.

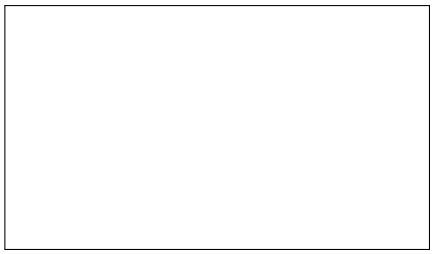

Figure 4: Placeholder for ChartQA accuracy vs. training set size $k \in \{10, 25, 50, 100, 200\}$ at N=3 iterations.

C.2 Per-Category Analysis on ChartQA

C.3 Distribution-Shift Adaptation: Time Series and Aggregates

Figure 3 in the main paper aggregates the distribution-shift experiment across backbones and stream constructions. This appendix shows the per-step trajectory along the stream (Figure 5) and the full per-backbone numerical aggregates (Table 7).

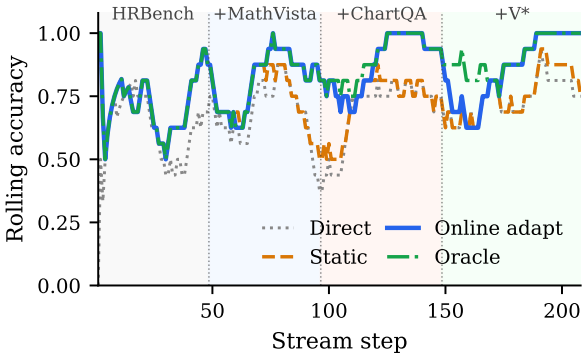

Figure 5: **Rolling accuracy along the natural stream (GPT-5.4).** Phase labels above the plot mark the newly introduced family at each shift; earlier families remain present in the mix. *Static* lags at each shift; *Online adapt* catches up to *Oracle* within a 2–7 case detection latency.

D Qualitative Analysis of Evolved Capabilities

This appendix gives three qualitative case studies of evolved capabilities. The examples are intended to make the generated artifacts concrete, not to introduce new quantitative results. Each case should be instantiated with the corresponding logged failure, generated skill/tool file, visual artifact, and validation record before submission. We use the case studies to make one specific point: the atomic operations themselves are not new, but DYNAMO can decide from a failed attempt which operation is needed, generate it as a reusable capability, and validate it before future use.

Replay protocol	Direct	Static	Online adapt	Oracle
GPT-4o (Full)				
Capability-relevant	0.418±0.020	0.558±0.023	0.789±0.025	0.818±0.022
Stress	0.068±0.009	0.347±0.011	0.912±0.008	0.961±0.010
Natural	0.664±0.032	0.673±0.030	0.702±0.028	0.705±0.028
o4mini (Full)				
Capability-relevant	0.467±0.019	0.626±0.019	0.852±0.019	0.881±0.016
Stress	0.047±0.006	0.412±0.006	0.940±0.009	0.984±0.007
Natural	0.745±0.029	0.787±0.028	0.807±0.027	0.808±0.027
GPT-5.4 (Full)				
Capability-relevant	0.479±0.021	0.636±0.020	0.870±0.020	0.899±0.018
Stress	0.026±0.004	0.391±0.004	0.947±0.007	0.995±0.005
Natural	0.761±0.029	0.800±0.029	0.830±0.026	0.833±0.025
Qwen3.5-27B (Full)				
Capability-relevant	0.514±0.015	0.667±0.016	0.888±0.020	0.917±0.018
Stress	0.122±0.007	0.488±0.007	0.940±0.009	0.984±0.008
Natural	0.821±0.026	0.847±0.023	0.861±0.022	0.862±0.022
Doubao-Seed-2.0 (Full)				
Capability-relevant	0.635±0.014	0.773±0.015	0.900±0.018	0.917±0.016
Stress	0.621±0.013	0.768±0.014	0.902±0.014	0.920±0.014
Natural	0.850±0.023	0.876±0.022	0.893±0.019	0.895±0.019

Table 7: **Per-backbone aggregates for the distribution-shift experiment.** Mean accuracy with seed std (50–200 seeds per cell). Bars in Figure 3 are computed from this table.

D.1 Case A: ReFocus-Style Skill and Tool for Structured Images

ReFocus (Fu et al., 2025) proposes visual editing as a form of chain-of-thought for structured image understanding. It asks an MLLM to generate Python code that edits the image, including drawing boxes, highlighting relevant regions, and masking distractors. A DYNAMO case belongs in this category when the generated capability explicitly changes the visual input shown to the solver.

Triggering failure. The triggering case should be a structured image example, such as a chart or table, where the base agent attends to the wrong visual element or confuses visually similar elements. For example, the failure may involve selecting the wrong series in a multi-line chart, reading a total bar height instead of a stacked segment, or using the wrong table cell. [Insert benchmark, case id, question, ground-truth answer, and base answer.]

Generated skill. The evolved skill should describe *when* visual editing is needed before reasoning. A typical ReFocus-style skill has the following structure:

```
## Skill: Edit-Then-Read for Structured Images

When-to-Use:
Use when the question refers to a specific chart/table element
that is visually close to distractors, such as one series
      among
many lines, one segment in a stacked bar, or one table
      row/column.

Strategy:
1. Identify the visual entity named by the question.
2. Before computing the answer, create an edited view that
      marks
   only the relevant region and suppresses likely distractors.
```

Case study	Generated capability	Prior motif	Claim supported
Case A: Structured-image editing	A skill that instructs the solver to isolate the relevant chart/table structure, plus a tool that highlights, boxes, or masks visual evidence.	ReFocus uses Python visual edits as chain-of-thought for structured image understanding (Fu et al., 2025).	DYNAMO can generate a reusable visual-editing capability from failure, rather than relying on a fixed visual-editing interface.
Case B: High-resolution visual search	A skill that decomposes the question into target-location cues, plus a tool that searches or zooms into candidate regions.	ZoomEye performs training-free tree-based image exploration over zoomed sub-regions (Shen et al., 2025).	DYNAMO can generate a local coarse-to-fine search capability without being given ZoomEye as a predefined module.
Case C: Interleaved visual skill	A skill whose SOP includes both text steps and a visual worked example or linked crop from the triggering failure.	Visual-thought methods show that intermediate visual artifacts can guide structured reasoning.	DYNAMO can store procedural and visual evidence together as a retrieved capability.

Table 8: **Three qualitative forms of evolved capability.** The case studies compare generated artifacts to prior visual-reasoning motifs. They do not claim that DYNAMO reproduces the full prior algorithms.

```
3. Re-read the edited image and extract the required value.
4. Perform the requested arithmetic or comparison.

Common Pitfalls:
- Do not answer from the unedited image if multiple marks are
  visually similar.
- Do not treat the visual edit as adding new information; it
      only
  exposes the evidence already present in the image.
```

Generated tool. The paired tool is a small image-editing program. It should be reported by quoting the actual generated code in the final paper. A representative form is:

```
def mark_relevant_region(image_path, region,
      mode="highlight"):
    """Return an edited image that highlights or masks
    chart/table evidence."""
    from PIL import Image, ImageDraw
    img = Image.open(image_path).convert("RGB")
    draw = ImageDraw.Draw(img, "RGBA")
    x1, y1, x2, y2 = region
    if mode == "highlight":
        draw.rectangle([x1, y1, x2, y2], outline=(255, 0, 0,
      255), width=6)
        draw.rectangle([x1, y1, x2, y2], fill=(255, 255, 0,
      60))
    elif mode == "mask_distractors":
        # final paper should quote the actual generated
      masking logic
        pass
    out_path = image_path.replace(".png", "_marked.png")
    img.save(out_path)
    return out_path
```

Why this matches ReFocus. Like ReFocus, the capability uses visual editing as an intermediate reasoning step: the model does not merely write a textual explanation, but creates an edited visual artifact that changes the evidence shown to the next solver call. The difference is that ReFocus defines visual editing as the reasoning interface, whereas DYNAMO produces this edit skill/tool only after a failure analysis decides that the bottleneck is visual disambiguation.

Evidence to insert. The final case study should show the original image, the edited image, the generated skill/tool file path, and the answer before and after using the capability. [Insert validation result and retry answer.]

D.2 Case B: ZoomEye-Style Skill and Tool for High-Resolution Search

ZoomEye (Shen et al., 2025) addresses the loss of fine visual detail by treating image understanding as a tree-based exploration process. The full image is the root node, zoomed image regions are child nodes, and the model searches over promising regions until it finds the visual evidence needed to answer. A DYNAMO case belongs in this category when the generated capability performs region selection, cropping, zooming, or coarse-to-fine search.

Triggering failure. The triggering case should involve a high-resolution image where the base agent knows the object or region type but cannot inspect it accurately from the full image. Examples include small text, distant signs, number plates, small object attributes, or a target object occupying a small fraction of the image. [Insert benchmark, case id, question, ground-truth answer, and base answer.]

Generated skill. The evolved skill should make the search procedure explicit:

```
## Skill: Coarse-to-Fine Region Search

When-to-Use:
Use when the question asks about a small object, distant text,
fine-grained attribute, or localized region in a
      high-resolution image.

Strategy:
1. Parse the question for target object, attribute, and
      spatial cues.
2. Inspect the full image only to estimate candidate regions.
```

```
3. Divide the image into coarse tiles and score each tile for
      relevance.
4. Zoom into the best candidate region and answer from the
      enlarged view.
5. If the crop is ambiguous, repeat with a finer crop around
      the target.

Common Pitfalls:
- Do not answer from the globally downsampled image when the
      requested
  evidence is small.
- Do not crop solely by image center; use question-specific
      cues.
```

Generated tool. The paired tool implements a local search/zoom operation. The final version should quote the actual generated code; a representative form is:

```
def coarse_to_fine_zoom(image_path, target_description,
      grid_size=4):
    """Return candidate high-resolution crops for a
    target described in text."""
    from PIL import Image
    img = Image.open(image_path).convert("RGB")
    w, h = img.size
    crops = []
    for gy in range(grid_size):
        for gx in range(grid_size):
            box = (gx*w//grid_size, gy*h//grid_size,
                   (gx+1)*w//grid_size, (gy+1)*h//grid_size)
            crop = img.crop(box)
            out = image_path.replace(".png",
      f"_tile_{gx}_{gy}.png")
            crop.save(out)
            crops.append((out, box))
    return crops
```

Why this matches ZoomEye. The similarity is the coarse-to-fine visual search motif. Both methods avoid forcing the VLM to answer from a single fixed-resolution full image. The difference is scope: ZoomEye is a general tree-search algorithm, while the DYNAMO capability is generated from a particular failure and may implement only the subset of zoom/search behavior needed for that task family.

Evidence to insert. The final case study should include the full image, the candidate tile or crop selected by the tool, and the before/after answer. [Insert selected crop, validation result, and retry answer.]

D.3 Case C: Interleaved Visual Skill

The third case is different from the previous two because the main artifact is not only an executable program. An interleaved visual skill is a retrieved Markdown skill that contains both textual reasoning instructions and visual evidence, such as an embedded crop, annotated chart, or linked processed image from the triggering failure.

Triggering failure. This case should involve a problem where a text-only SOP is underspecified because the visual pattern itself matters. A common example is chart reasoning: the agent may know that it should subtract segment boundaries in a stacked bar chart, but still confuse which boundary belongs to the target segment. [Insert benchmark, case id, question, ground-truth answer, and base answer.]

Generated skill. The generated skill should be quoted directly if available. A representative interleaved form is:

```
## Skill: Stacked-Bar Segment Reading with Visual Anchor

When-to-Use:
Use when the question asks for a value, difference, or
      comparison
involving a segment of a stacked bar chart.

Strategy:
1. Locate the target bar using the x-axis label.
2. Locate the target segment using the legend colour.
3. Read the lower and upper boundaries of the segment.
4. Subtract lower from upper to obtain the segment value.
5. Verify the result against the visual anchor below.

Visual Anchor:
[embedded crop or linked image showing the annotated segment
      boundaries]

Common Pitfalls:
- Do not read the total bar height when the question asks for
      one segment.
- Do not use a nearby segment with a visually similar colour.
```

Optional generated tool. In this case, the paired tool may be simple: it produces the crop or annotation that is embedded in the skill.

```
def make_visual_anchor(image_path, crop_box, annotations):
    """Create an annotated crop used as the visual anchor in
      a skill."""
    from PIL import Image, ImageDraw
    img = Image.open(image_path).convert("RGB")
    crop = img.crop(crop_box)
    draw = ImageDraw.Draw(crop)
    for ann in annotations:
        draw.rectangle(ann["box"], outline=ann.get("color",
      "red"), width=4)
        if "label" in ann:
            draw.text((ann["box"][0], ann["box"][1]),
      ann["label"],
                      fill=ann.get("color", "red"))
    out_path = image_path.replace(".png",
      "_visual_anchor.png")
    crop.save(out_path)
    return out_path
```

Why this is interleaved. The retrieved capability is not purely textual and not purely executable. The skill tells the agent how to reason, while the visual anchor shows what the relevant visual configuration looks like. This is related to visual-thought methods, but the visual artifact is stored persistently inside the capability library and retrieved on later cases.

Evidence to insert. The final case study should include the generated Markdown skill, the visual anchor image, and a before/after answer showing how the retrieved interleaved skill changes the solver's behavior. [Insert validation result and retry answer.]

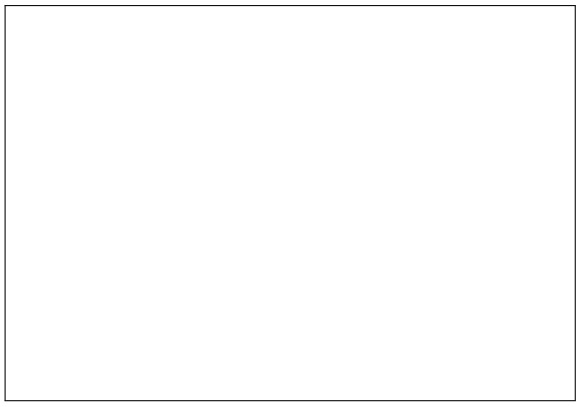

Figure 6: **Placeholder for a concrete before/after case study.** The final version should insert the original image, the failed base answer, the generated skill/tool artifact, the processed visual output if any, and the validated retry answer.

E Case Study Artifact Checklist

To avoid conflating qualitative analysis with unreported experiments, we instantiate each case study only from artifacts produced by the evolution loop. For each of the three cases above, the final appendix should include:

- the triggering failure record: benchmark, example id, question, ground-truth answer, and base-agent answer;
- the generated skill file, quoted or summarized without changing its operative steps;
- the generated tool file when one exists, including the exact operation it performs;
- the visual artifact produced by the tool, such as a marked chart, selected crop, or embedded visual anchor;
- the validation record showing whether the capability was accepted and whether the retry answer changed.

If a case lacks one of these artifacts, we mark the missing field explicitly rather than inferring it from the intended behavior. This keeps the case studies as evidence for the form of the evolved capabilities, while the quantitative claims remain grounded in the tables in Section 4.

F Reproducibility Statement

The code, evolved capability files (`learned/` directory), and all evaluation scripts will be released in an anonymised repository upon acceptance. The experiments use publicly available benchmark datasets and the VLM backbones described in Section 4.1. Evolved capability sets (`*.skill.md` and `*.tool.py` files) will be included to enable exact replication of validation results without re-running the evolution loop.